МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра квантовых информационных технологий, практической и прикладной информатики (ККИТПиПИ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №4
Выполнение задач обработки данных
в распределённых вычислениях

Студент группы ИНБО-12-23, Албахтин И.В. _____ (подпись)

Преподаватель Приходько Н.А. _____ (подпись)

Москва 2026 г.

Содержание

1 Цель и постановка экспериментов 3
2 Стенд и данные 3
3 Ленивое выполнение и входное разбиение 4
4 Repartition, coalesce и shuffle 4
5 Сравнение стратегий JOIN 6
6 Persist и стоимость повторного использования 8
7 Benchmark, shuffle partitions и small files 9
8 Самостоятельный вариант B 10
8.1 Экспериментальные выводы варианта B 13
9 Общие выводы и воспроизводимость 13
Ответы на контрольные вопросы 14
Источники 21

1 Цель и постановка экспериментов

Цель работы — исследовать физическое выполнение преобразований Spark: ленивые вычисления, партиции и задачи, shuffle, стратегии JOIN, сохранение промежуточного результата, повторные измерения и создание part-файлов. Учебные опыты выполняются на Olist; самостоятельный вариант B использует order_items и sellers с группировкой по seller_state.

Полный Run All после запуска нового ядра завершён 26.10.2026 00:02:19 МСК. Последовательно выполнены 12 кодовых ячеек, ошибок 0; длительность с автоматическим сохранением Spark UI — 45,75 с. В последней ячейке выполнен spark.stop().

2 Стенд и данные

Результаты приведены в таблице 1.

Таблица 1 – Паспорт экспериментального стенда

Параметр	Значение
VM / ОС	TIABD-Albakhtin-IV / Ubuntu Server 22.04.5 LTS
CPU / память	2 vCPU / 4 ГБ RAM
Python / Spark	3.10.12 / 3.5.7
Java	openjdk version "17.0.20.1" 2026-08-18
Master / parallelism	local[*] / 2
Driver memory	1536m
Shuffle partitions / AQE	8 / false
Auto broadcast threshold	-1

Методичка использует Python 3.11; на стенде применяется совместимый с Spark 3.5.7 Python 3.10.12 и JDK 17. Local[*] сохраняет модель распределённых данных, но выполняет задачи на одном узле. Подготовка SQLite, выгрузка CSV и запись исходного Parquet исключены из измерений. Сравниваемые варианты выполняются в одной SparkSession.

Ecommerce.sqlite восстановлена из исходных CSV Olist предыдущей практики. Исходный файл SQLite примера не предоставлялся. Восстановленная база содержит 100 000 отзывов вместо 97 621 в примере ПР3; отзывы в опытах ПР4 не используются. Для всех пяти используемых таблиц число строк совпало с методичкой. Справочники products и sellers проверены на уникальность ключей и отсутствие несопоставленных позиций заказа.

Результаты приведены в таблице 2.

Таблица 2 – Входные DataFrame из Parquet

Таблица	Строки	Партиции
geolocation	1 000 163	2
order_items	112 650	2
products	32 951	1
sellers	3 095	1
product_category_name_translation	71	1

3 Ленивое выполнение и входное разбиение

```
prepared = (geo.select("geolocation_state", "geolocation_lat")
   .filter(F.col("geolocation_state").isNotNull())
   .withColumn("abs_lat", F.abs("geolocation_lat")))
rows = prepared.count()
```

До создания цепочки и после transformations список Jobs группы пуст: [] и []. После count появились Jobs [32]; получено 1 000 163 строк за 0,068 с. Это подтверждает отложенное выполнение. Count не использует abs_lat, поэтому оптимизатор вправе исключить вычисление этого поля; время не трактуется как стоимость всех выражений цепочки.

Результаты приведены в таблице 3.

Таблица 3 – Распределение входных партиций geolocation

Раздел	Строки
0	534 580
1	465 583

Результаты приведены в таблице 4.

Таблица 4 – Наиболее частые штаты

Штат	Строки	Доля %
SP	404 268	40,42
MG	126 336	12,63
RJ	121 169	12,11
RS	61 851	6,18
PR	57 859	5,78

4 Repartition, coalesce и shuffle

Сначала geolocation разбита на 16 партиций по geolocation_state, затем число разделов уменьшено до четырёх двумя способами. В профиле явно учитываются все разделы, включая пустые: среднее равно числу строк, делённому на полное число партиций. RoundRobin repartition

перераспределяет строки; coalesce объединяет существующие разделы и сохраняет часть перекоса.

Результаты приведены в таблице 5.

Таблица 5 – Нагрузка разделов после переразбиения

Способ	Разделы	Минимум	Максимум	Среднее
skewed16	16	0	466 119	62 510,19
repartition4	4	250 040	250 041	250 040,75
coalesce4	4	88 462	626 485	250 040,75

```
skewed16 = geo.repartition(16, "geolocation_state")
repart4 = skewed16.repartition(4)
coal4 = skewed16.coalesce(4)
# План: Exchange RoundRobinPartitioning(4) либо Coalesce 4
```

Coalesce не добавляет новый полный shuffle, но не отменяет исходный Exchange, создавший skewed16. Поэтому ненулевые Shuffle Read/Write у всего действия coalesce не противоречат устройству этого оператора. В seller_stats частичная HashAggregate уменьшает данные до Exchange hashpartitioning(seller_id, 8), затем выполняется финальная агрегация.

Результаты приведены в таблице 6.

Таблица 6 – Пять продавцов по сумме price

seller_id	Позиций	Выручка R$	Средняя R$
4869f7a5	1 156	229 472,63	198,51
53243585	410	222 776,05	543,36
4a3ca931	1 987	200 472,92	100,89
fa1c13f2	586	194 042,03	331,13
7c67e144	1 364	187 923,89	137,77

В таблице показаны первые восемь символов seller_id; полные ключи сохранены в результатах. Scan читает Parquet, Filter отбирает price ≥ 100, Project формирует gross, Exchange меняет распределение, HashAggregate завершает агрегацию. Эти операторы выделены также в отдельном плане operator_demo.txt.

Результаты приведены в таблице 7.

Таблица 7 – Журнал выполненных действий Spark UI

Опыт	Jobs	Stages	Tasks	Read КиБ	Write КиБ
GroupBy	1	2	10	262,96	262,96
Repartition	1	4	23	172,58	172,58
Coalesce	1	3	7	151,84	151,84
SMJ	1	4	12	3 825,50	3 825,50
BHJ	2	3	4	0,12	0,12

Опыт	Jobs	Stages	Tasks	Read КиБ	Write КиБ
Заполнение кэша	3	4	5	0,12	0,12
Повтор из кэша	1	3	11	6,29	6,29

Для измеренного groupBy стадии и число задач: 91: 2; 92: 8. Суммы счётчиков независимым скриптом сопоставлены с сохранённым REST API; номер стадии относится только к текущему приложению.

5 Сравнение стратегий JOIN

Один INNER JOIN order_items–products по product_id выполнен как SMJ и BHJ. Оба результата содержат 112 650 строк. Проверки ключей исключили размножение фактов и потерю несовпавших позиций. SMJ использует распределение по ключу и сортировку, BHJ — BroadcastExchange справочника и BroadcastHashJoin.

```
smj = items.hint("merge").join(
    products.hint("merge"), "product_id", "inner")
bhj = items.join(F.broadcast(products), "product_id", "inner")
```

Результаты приведены в таблице 8.

Таблица 8 – Повторные измерения JOIN

Стратегия	Первый с	t1 с	t2 с	t3 с	Медиана с
SMJ	0,243	0,178	0,153	0,144	0,153
BHJ	0,226	0,134	0,094	0,082	0,094

Небольшой shuffle финального count может сохраняться и у BHJ; он не равен перераспределению большой стороны по product_id. Поэтому сравнение учитывает вид Exchange в плане. Скриншоты ниже показывают фрагменты SQL/DataFrame запросов с узлами JOIN; полные текстовые планы сохранены отдельно.

На рисунке 1 показан результат выполнения на стенде.

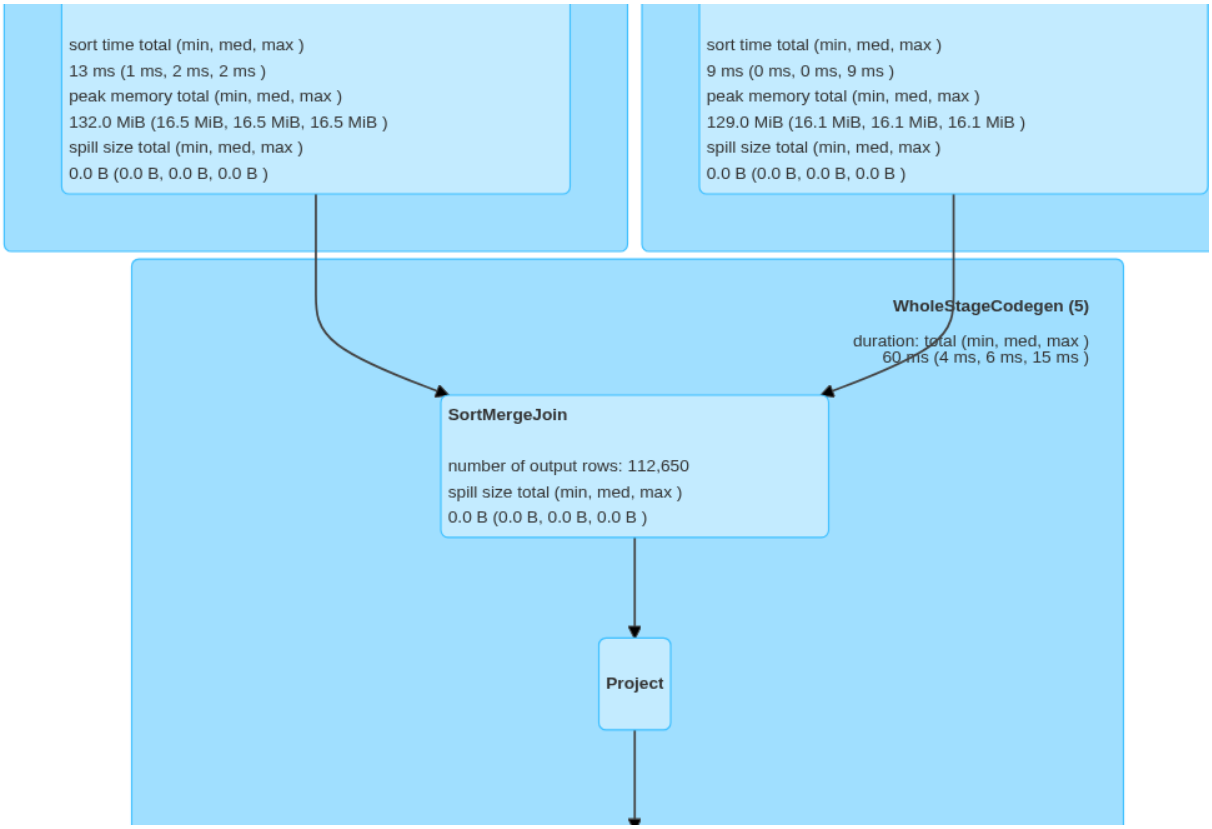


Рисунок 1 – SQL/DataFrame: узел SortMergeJoin выполненного запроса

На рисунке 2 показан результат выполнения на стенде.

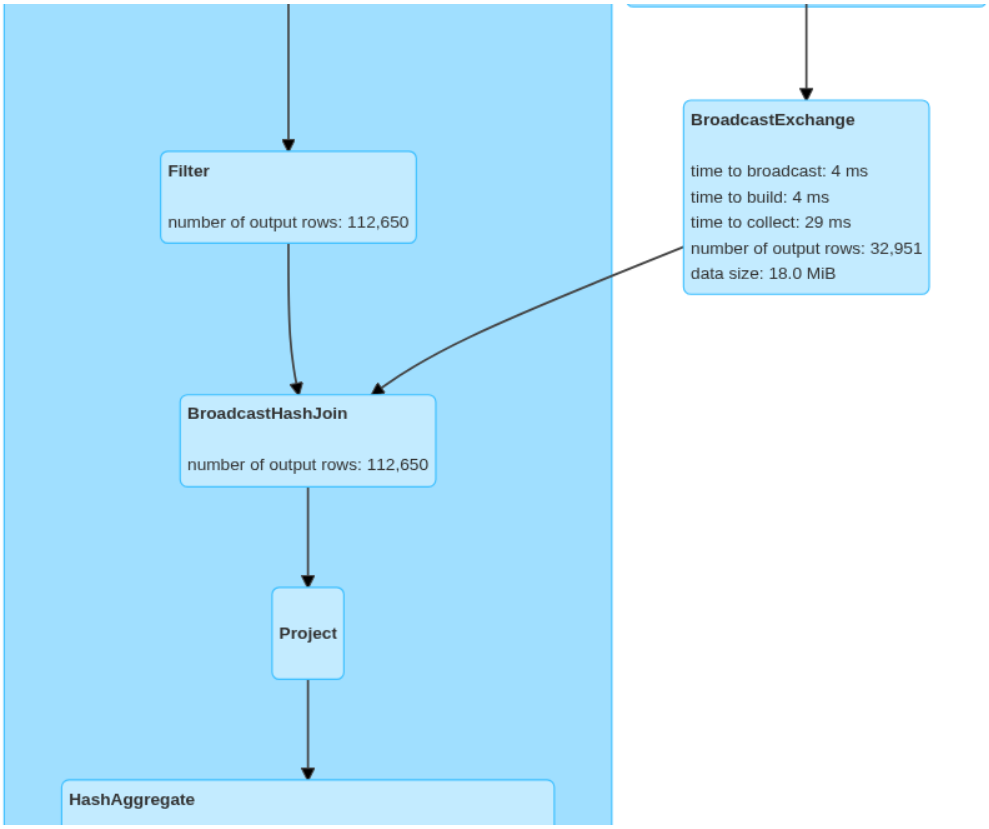


Рисунок 2 – SQL/DataFrame запроса BHJ с передачей справочника

6 Persist и стоимость повторного использования

Промежуточный DataFrame объединяет позиции, категории и продавцов; gross_value = price + freight_value. Использован persist(StorageLevel.MEMORY_AND_DISK). Два действия суммируют gross_value по категории и по штату с последующим получением контрольной суммы. Это заставляет вычислить используемые значения, а не только число строк.

Результаты приведены в таблице 9.

Таблица 9 – Материализация и два одинаковых действия

Вариант	Заполнение с	Action 1 с	Action 2 с	Всего с
Без persist	—	0,313	0,253	0,567
С persist	0,527	0,149	0,079	0,755

С материализацией кэш не окупился за два действия: 0,755 с против 0,567 с. Все четыре контрольные суммы совпадают с независимой суммой SQLite 15 843 553,24 с допуском 0,01. После unpersist(blocking=True) is_cached=false, список сохранённых блоков Storage пуст.

На рисунке 3 показан результат выполнения на стенде.

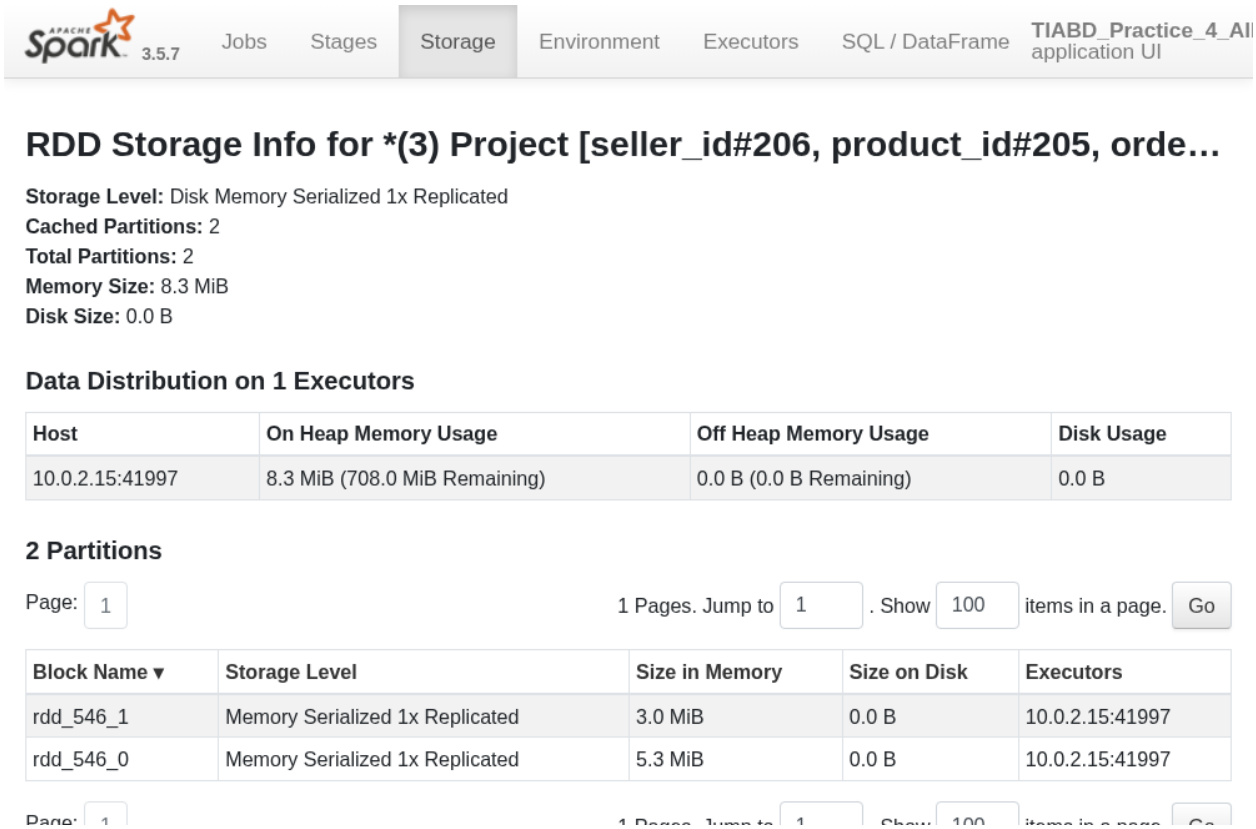


Рисунок 3 – Материализованный MEMORY_AND_DISK до unpersist

7 Benchmark, shuffle partitions и small files

N=2 000 000, key=id%1000, value=sqrt(id+1). После общего прогрева для p=1,2,4,8,16,32 выполнены по три измерения в одной сессии при shuffle.partitions=8. Action вычисляет сумму агрегированных значений; все контрольные суммы проверены через независимый math.fsum с учётом точности double.

Результаты приведены в таблице 10.

Таблица 10 – Влияние входного p на время

p	t1 c	t2 c	t3 c	Медиана c	T1/Tp
1	0,099	0,070	0,051	0,070	1,000
2	0,121	0,089	0,083	0,089	0,793
4	0,093	0,078	0,081	0,081	0,871
8	0,143	0,139	0,078	0,139	0,506
16	0,150	0,094	0,098	0,098	0,716
32	0,161	0,136	0,138	0,138	0,509

Наименьшая медиана серии получена при p=1: 0,070 с. Показатель T1/Tp описывает изменение числа разделов на том же узле, а не ускорение от добавления физических узлов.

Результаты приведены в таблице 11.

Таблица 11 – Отдельный опыт с числом shuffle partitions

q	Группы	Время c
2	27	0,299
4	27	0,221
8	27	0,244
16	27	0,271
32	27	0,262

В этом дополнительном опыте вход зафиксирован на восьми разделах. Изменяется q, а не число строк и не число CPU. Одиночные измерения q не используются для устойчивого рейтинга производительности.

Результаты приведены в таблице 12.

Таблица 12 – Запись order_items в Parquet

p	part-файлы	Объём МиБ	Средний КиБ
1	1	6,456	6 611,425
4	4	7,464	1 910,897
16	16	8,756	560,407
64	64	10,301	164,824

Количество и размеры part-файлов сверены с файловой системой. Coalesce(1) отдельно создаёт один раздел записи и ограничивает её параллелизм. Дополнительное включение AQE для группировки geolocation дало 27 групп и 1 выходную партицию при исходной настройке 32. Итоговый план содержит AdaptiveSparkPlan; после опыта восстановлены AQE=false и shuffle.partitions=8.

8 Самостоятельный вариант B

Из Parquet прочитаны 112 650 позиций и 3 095 продавцов; исходное число партиций — 2 и 1. Select, filter(price>0) и withColumn(gross) не создали Jobs до Action. После действия появилась группа Jobs [103]. Обе стратегии JOIN по seller_id сохранили 112 650 строк.

Результаты приведены в таблице 13.

Таблица 13 – Агрегация по штату продавца

Штат	Позиции	Цена с доставкой R$
SP	80 342	10 235 883,88
PR	8 671	1 458 900,73
MG	8 827	1 224 159,80
RJ	4 818	937 814,12
SC	4 075	738 973,13

Все 23 групп независимо совпали с SQLite. Штат относится к продавцу, а не покупателю. GroupBy варианта B выполнил 27 задач, Shuffle Read/Write — 5,20/3,39 КиБ. План включает BroadcastHashJoin и Exchange по seller_state.

Результаты приведены в таблице 14.

Таблица 14 – Баланс четырёх разделов варианта B

Способ	Минимум	Максимум	Среднее
repartition	28 161	28 165	28 162,50
coalesce	5 348	88 381	28 162,50

Результаты приведены в таблице 15.

Таблица 15 – JOIN самостоятельной части

Стратегия	Время с	Stages	Tasks	Read/Write КиБ
SMJ	0,166	4	12	1 337,59/1 337,59
BHJ	0,092	3	4	0,12/0,12

Результаты приведены в таблице 16.

Таблица 16 – Два действия самостоятельной части

Вариант	Заполнение с	Action 1 с	Action 2 с	Всего с
Без persist	—	0,178	0,123	0,300
С persist	0,198	0,117	0,163	0,478

Контрольные суммы двух группировок до и после кэша совпали с SQLite. Кэш не окупился за эти два действия с учётом материализации. После unpersist сохранённые блоки удалены.

Результаты приведены в таблице 17.

Таблица 17 – Прогрев и три повтора для четырёх p

p	t1 с	t2 с	t3 с	Медиана с
1	0,147	0,133	0,117	0,133
2	0,143	0,139	0,127	0,139
4	0,126	0,126	0,147	0,126
8	0,145	0,219	0,149	0,149

Результаты приведены в таблице 18.

Таблица 18 – Две записи итогового агрегата

p	Файлы	Объём КиБ
1	1	1,335
4	4	4,168

На рисунке 4 показан результат выполнения на стенде.

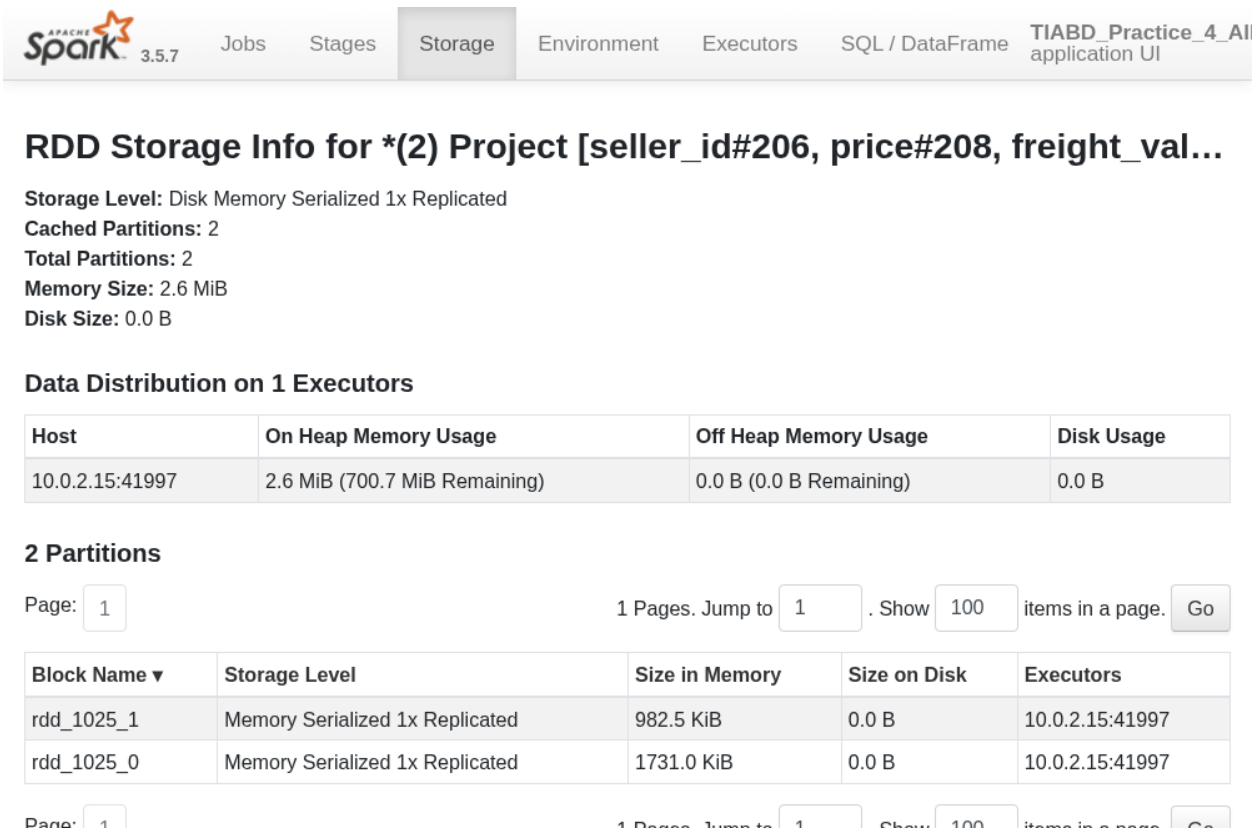


Рисунок 4 – Storage самостоятельного варианта B до освобождения блоков

На рисунке 5 показан результат выполнения на стенде.

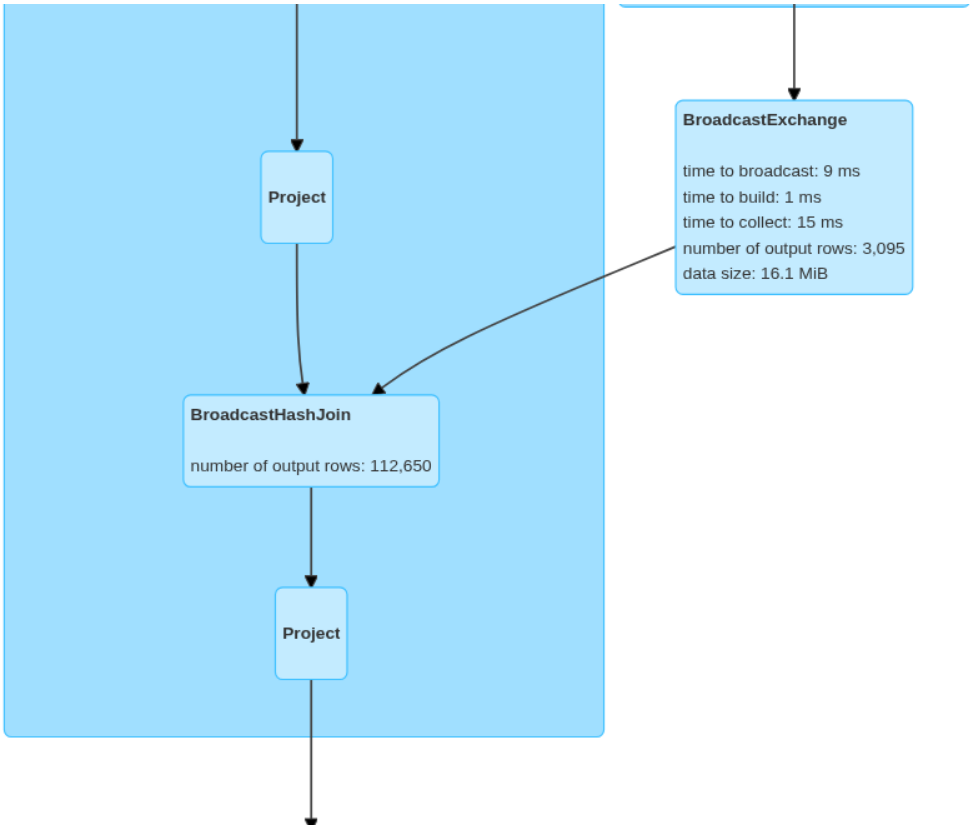


Рисунок 5 – SQL/DataFrame самостоятельного соединения с sellers

На рисунке 6 показан результат выполнения на стенде.

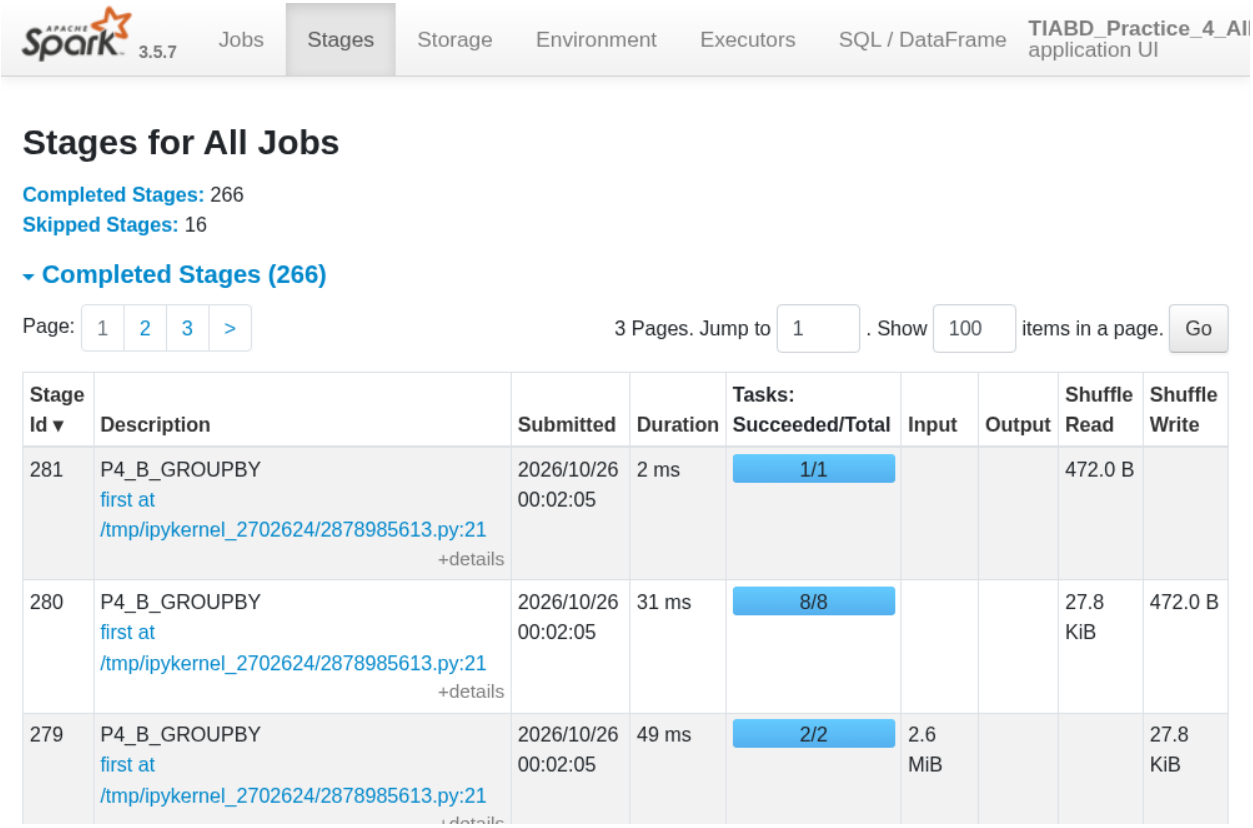


Рисунок 6 – Завершённые стадии и задачи самостоятельного варианта B

8.1 Экспериментальные выводы варианта B

1. Цепочка из трёх transformations не создала Job до Action; после действия обработано 112 650 строк.

2. После repartition(4) максимум нагрузки равен 28 165 строк, после coalesce(4) — 88 381. Баланс зависит от способа распределения, а не только от числа разделов.

3. Обе стратегии JOIN дали 112 650 строк. SMJ использовал сортировку и shuffle, BHJ — передачу справочника sellers. Времена относятся к одиночным действиям данного опыта.

4. Полная стоимость кэша для двух действий составила 0,478 с против 0,300 с без кэша; цена материализации учитывается отдельно.

5. Среди четырёх проверенных p наименьшая медиана получена при p=4: 0,126 с. Это наблюдение для текущей связки и ресурсов VM.

6. При записи агрегата с одним и четырьмя разделами получены соответственно один и четыре part-файла; итоговый набор строк не изменился.

Ограничения самостоятельного опыта: небольшой справочник и агрегат не воспроизводят память и сетевые затраты крупного кластера; короткие действия на двух vCPU чувствительны к прогреву JVM, файловому кэшу и фоновой нагрузке. Одиночные измерения JOIN не заменяют повторную серию для устойчивого вывода о скорости.

9 Общие выводы и воспроизводимость

1. Transformations строят план; Action запускает необходимое выполнение, что подтверждено журналами обеих серий.

2. Входные партиции, shuffle partitions и CPU — разные сущности. Число задач меняется между стадиями одного Job.

3. GroupBy содержит частичную и финальную агрегацию с Exchange; счётчики UI показывают фактический обмен.

4. Repartition перераспределяет строки, coalesce объединяет разделы; одинаковое выходное p не гарантирует одинаковый баланс.

5. SMJ и BHJ сохранили логический результат и использовали разные механизмы передачи и сортировки данных.

6. Persist оценивается вместе с материализацией; освобождение блоков подтверждено после unpersist.

7. Медианы трёх повторов не дают универсального правила о лучшем p. На локальном узле накладные расходы могут перекрыть выгоду дополнительных задач.

8. Число и размер part-файлов измерены после write. Число разделов для вычисления и для сохранения результата выбирается по разным ограничениям.

9. AQE изменил фактическое выходное разбиение после получения статистики; основной опыт выполнен с AQE=false для контроля условий.

Ограничения общей серии: два vCPU одной VM не моделируют физическую сеть кластера; состояние JVM, файлового кэша и хоста влияет на короткие измерения. Относительное ускорение по числу партиций не является оценкой масштабирования по серверам.

```
cd /home/albakhtin/tiabd/АлбахтинИВ/Spark
/home/albakhtin/tiabd/.venv/bin/python run_pr4.py
/home/albakhtin/tiabd/.venv/bin/python verify_pr4_data.py
```

Notebook, протокол Run All, JSON измерений, полные планы, REST API, снимки UI и итоговый агрегат сохранены в личном каталоге Spark. Снимки получены автоматически между опытами до spark.stop(); время их создания не включено в отдельные замеры действий.

Ответы на контрольные вопросы

1. Чем transformation отличается от action с точки зрения момента фактического выполнения?

Transformation описывает преобразование данных и строит план. Например, select, filter и withColumn создают новый DataFrame, но не требуют сразу вычислить все его строки. Action требует результат: count, collect, show или запись запускают необходимое выполнение плана.

2. Что означает lazy evaluation и почему замер времени вокруг withColumn() без Action некорректен?

Lazy evaluation — отложенное выполнение вычислений до момента, когда нужен результат. Вызов withColumn("gross", price + freight_value) прежде всего добавляет выражение в план. Время этого вызова не включает полное чтение и обработку набора.

3. Как связаны Job, Stage и Task?

Job объединяет вычислительную работу, необходимую для получения результата действия. Планировщик разбивает Job на Stages по зависимостям; границы часто возникают при shuffle. Внутри стадии запускаются Tasks, каждая из которых обычно обрабатывает одну её партицию.

4. Почему число Tasks в разных Stages одного Job может отличаться?

Каждая стадия работает со своим разбиением. Первая читает входные партиции, следующая может обрабатывать восемь shuffle partitions, а финальная глобальная агрегация — один раздел. Число задач определяется партициями конкретной стадии, а не постоянным числом на весь Job.

5. Что такое partition и чем он отличается от физического CPU core?

Partition — логическая часть распределённого набора данных. CPU core — аппаратный ресурс, на котором выполняются вычисления. У одной партиции нет постоянного собственного ядра: планировщик назначает задачи доступным вычислительным слотам.

6. Почему local[*] сохраняет модель распределённого выполнения, хотя используется один компьютер?

local[*] использует доступные Spark логические процессоры одного компьютера для выполнения задач потоками локальной JVM. DataFrame всё равно разбит на партиции; сохраняются Driver, граф зависимостей, Stages, Tasks и обмен данными между разделами.

7. Что называют narrow и wide transformation? Приведите примеры.

При narrow transformation выходной раздел зависит от ограниченного числа входных разделов, и полное перераспределение не требуется. Типичные примеры — filter, select, простые withColumn и уменьшение разбиения через coalesce без shuffle.

8. Что такое shuffle и почему он обычно дороже локального filter()/select()?

Shuffle перераспределяет записи между партициями, например по значению seller_id. Это требует подготовки блоков, сериализации, записи промежуточных данных и их чтения другими задачами; на физическом кластере добавляется сетевой обмен.

9. Как оператор Exchange в physical plan связан с shuffle?

Exchange обозначает изменение распределения данных. Exchange hashpartitioning(seller_id, 8) собирает одинаковые ключи в согласованных разделах и связан с shuffle. Exchange RoundRobinPartitioning(4) перераспределяет строки по четырём разделам.

10. В чём различие repartition() и coalesce()?

repartition(n) обычно выполняет shuffle и формирует новое распределение из n разделов. Он позволяет увеличивать и уменьшать их количество. Без ключа часто получается более равномерное распределение; при разбиении по сильно перекошенному ключу баланс не гарантирован.

11. Почему coalesce() может сохранить неравномерные размеры partitions?

coalesce объединяет существующие разделы, а не распределяет каждую строку заново. Если один исходный раздел уже очень велик, его объединение с соседними не устраняет причину перекоса. Одинаковое число выходных разделов не означает одинаковый объём работы.

12. Что такое data skew и как он проявляется на geolocation_state?

Data skew — неравномерное распределение данных по ключам или партициям. Наиболее частый штат SP содержит 404 268 из 1 000 163 строк геолокации, около 40,42 %. При hash-разбиении по штату все строки SP попадают в один раздел с этим ключом.

13. Что задаёт spark.sql.shuffle.partitions и почему это не то же самое, что число partitions исходного Parquet?

Параметр задаёт базовое число разделов для типичных shuffle-операций Spark SQL, например агрегаций и соединений, требующих перераспределения. Он не переписывает исходные Parquet-файлы и не устанавливает число разделов их чтения.

14. Какие physical operators необходимо уметь распознавать в explain("formatted")?

Scan читает источник, Filter отбирает строки, Project выбирает или вычисляет поля. HashAggregate выполняет агрегацию, Exchange меняет распределение, Sort сортирует строки. Для JOIN важны SortMergeJoin, BroadcastExchange и BroadcastHashJoin.

15. Почему Sort Merge Join часто содержит Exchange и Sort с обеих сторон?

Sort Merge Join сопоставляет отсортированные потоки по ключу. Для этого совпадающие ключи обеих таблиц должны оказаться в согласованных партициях, а строки внутри них — в подходящем порядке. Exchange обеспечивает распределение, Sort — порядок.

16. При каком условии Broadcast Hash Join может быть выгоден?

Малая сторона должна быть достаточно небольшой, чтобы её передачу и размещение хеш-структуры могли выдержать исполнители. Тогда большая таблица соединяется с локальной копией справочника и избегает shuffle по ключу JOIN.

17. Почему большую таблицу нельзя безусловно broadcast-ить?

Broadcast создаёт передаваемое представление таблицы и копии данных у исполнителей. Хеш-структура может занимать больше памяти, чем сжатый исходный файл. При большом объёме возрастают стоимость передачи, риск тайм-аута и нехватки памяти.

18. Почему одинаковое число строк результата важно при сравнении двух JOIN-стратегий?

Сравнение производительности имеет смысл, когда запросы решают одну логическую задачу. Если один потерял несовпавшие факты или размножил их из-за дубликатов справочника, меньшая длительность не означает лучшую реализацию того же JOIN.

19. Что происходит при первом Action после persist()?

persist регистрирует намерение сохранять вычисленные разделы с выбранным уровнем хранения. Первый Action выполняет нужную часть lineage и заполняет кэш по мере вычисления. Action, читающий только часть набора, не обязан материализовать все партиции.

20. Почему cache() может ухудшить одноразовую задачу?

Кэширование добавляет расходы на сохранение блоков и занимает память, а выгода возникает при повторном использовании. Если набор нужен один раз, эти расходы могут не окупиться. Кроме того, кэш способен вытеснить полезные данные или усилить давление на память.

21. Для чего нужен unpersist()?

unpersist() удаляет регистрацию сохранения и освобождает блоки промежуточного набора. Это возвращает ресурсы другим вычислениям и не позволяет одному опыту случайно влиять на следующий. Сам DataFrame и его план при этом остаются доступными для последующего вычисления.

22. Почему единичный benchmark-run недостаточен и зачем использовать медиану?

Первый запуск может включать прогрев JVM, JIT-компиляцию, загрузку классов и холодное чтение. Последующие зависят от файлового кэша и фоновой нагрузки. Поэтому единичное время легко принять за свойство алгоритма, хотя оно отражает состояние системы.

23. Какие факторы делают абсолютное время Spark-задачи несопоставимым между разными компьютерами?

Влияют производительность и число ядер CPU, память, диск, виртуализация, операционная система, версии Java и Spark, параметры сессии, размер данных и фоновая нагрузка. Имеет значение и состояние JVM и файлового кэша.

24. Почему слишком большое число partitions увеличивает overhead?

Каждая задача требует планирования, запуска, передачи служебных данных и фиксации результата. Когда разделы становятся очень маленькими, эти расходы составляют значительную долю времени. На двух vCPU дополнительные десятки разделов также образуют очередь задач.

25. Как связаны output partitions, Tasks записи и число part-файлов?

Каждая выходная задача обычно записывает данные своей партиции в отдельный part-* файл. Для простого непустого набора без разбиения по каталогам число файлов часто совпадает с числом выходных разделов.

26. Почему coalesce(1) не является универсальным способом борьбы с small files?

Один раздел оставляет одну задачу записи и ограничивает параллелизм. Для большого результата это создаёт длительный последовательный участок и концентрирует нагрузку на одном исполнителе. Удобство одного файла не означает хорошую производительность.

27. Что меняет Adaptive Query Execution и почему основные опыты этой работы выполняются с AQE=false?

AQE использует статистику, полученную во время выполнения, чтобы корректировать физический план. Среди возможностей — объединение небольших shuffle partitions, обработка некоторых перекошенных соединений и смена JOIN-стратегии при подходящих условиях.

28. Какие вкладки Spark UI наиболее полезны для анализа groupBy, JOIN и cache?

Jobs связывает действие с выполненными заданиями. Stages показывает задачи, длительности, Shuffle Read/Write и другие счётчики. SQL/DataFrame раскрывает физический план запроса и метрики его узлов. Storage показывает материализованные сохранённые наборы и объём блоков.

29. В чём различие между логическим выводом о данных и экспериментальным выводом о вычислительной системе?

Логический вывод описывает содержание результата: например, какой штат продавцов имеет наибольшую сумму цены с доставкой. Экспериментальный вывод описывает получение результата: какие операторы, стадии и объёмы обмена потребовались при конкретных настройках.

30. Как сформулировать корректный вывод о производительности так, чтобы не превратить локальный результат в универсальное правило?

Нужно указать данные, стенд, фиксированные настройки, изменяемый фактор, способ измерения и наблюдаемый результат. Затем связать время с механизмом физического плана и UI и назвать существенные ограничения опыта.

Источники

1. Практическая работа №4. Выполнение задач обработки данных в распределённых вычислениях. Предоставленная методичка.

2. Olist: восстановленная ecommerce.sqlite и исходные CSV предыдущих практик; контрольные суммы в source_manifest.json.

3. Apache Spark 3.5.7. Monitoring and Instrumentation. https://spark.apache.org/docs/3.5.7/monitoring.html

4. Apache Spark 3.5.7. SQL Performance Tuning. https://spark.apache.org/docs/3.5.7/sql-performance-tuning.html